

EXPERIMENT NO. 6 - MongoDB

Name of Student	Sonam Chhabaidiya
Class Roll No	D15A_09
D.O.P.	
D.O.S.	
Sign and Grade	

AIM: To study CRUD operations in MongoDB

PROBLEM STATEMENT:

Create a database, create a collection, insert data, query and manipulate data using various MongoDB operations.

- Create a database named "inventory".
- Create a collection named "products" with the fields: (ProductID, ProductName, Category, Price, Stock).
- Insert 10 documents into the "products" collection.
- Display all the documents in the "products" collection.
- Display all the products in the "Electronics" category.
- Display all the products in ascending order of their names.
- Display the details of the first 5 products.
- Display the categories of products with a specific name.
- Display the number of products in the "Electronics" category.
- Display all the products without showing the "_id" field.
- Display all the distinct categories of products.
- Display products in the "Electronics" category with prices greater than 50 but less than 100.
- Change the price of a product.
- Delete a particular product entry.

THEORY:

1. Describe some of the features of MongoDB?

- **Document-Oriented:** Stores data as flexible, JSON-like documents (BSON).
- **Flexible Schema:** No fixed structure, supports dynamic data.
- **Horizontal Scalability:** Uses sharding to manage large datasets.
- **Replication:** Ensures high availability with replica sets.
- **Indexing:** Supports various indexes for faster query execution.
- **Aggregation Framework:** Provides powerful data processing using pipelines.
- **Ad-hoc Queries:** Enables complex queries with ease.

2. What are Documents and Collections in MongoDB?

Documents: JSON-like records storing data in key-value pairs. Example:

```
{
  "_id": "101",
  "name": "Alice",
  "age": 28,
  "email": "alice@example.com"
}
```

Collections: A group of documents, equivalent to tables in relational databases. They don't enforce strict schemas, allowing flexibility.

3. When to use MongoDB?

- Big Data Applications: Efficient for large, unstructured data.
- E-commerce Platforms: Ideal for product catalogs with dynamic attributes.
- Content Management Systems (CMS): Supports frequent changes in data models.
- Real-Time Analytics: Processes and analyzes data rapidly.
- IoT and Mobile Apps: Manages sensor data and app data effectively.
- Social Networks: Scales well for user-generated content.

4. What is Sharding in MongoDB?

Sharding: Distributes data across multiple servers to handle large datasets.

Shard Key: A field in documents used to split data across shards.

Components:

- **Shards:** Store actual data.
- **Config Servers:** Maintain metadata and sharding configuration.
- **Mongos:** Routes queries to the appropriate shards.

Benefits:

- Supports large-scale data management.
- Improves read and write performance.
- Ensures fault tolerance and high availability.

GITHUB LINK: <https://github.com/sonamcc/webx65>

TYPE 1:

OUTPUT:

Insert Data (Create Operation)

1. Open your `inventory` collection.
2. Click "Insert Document" (top-right).

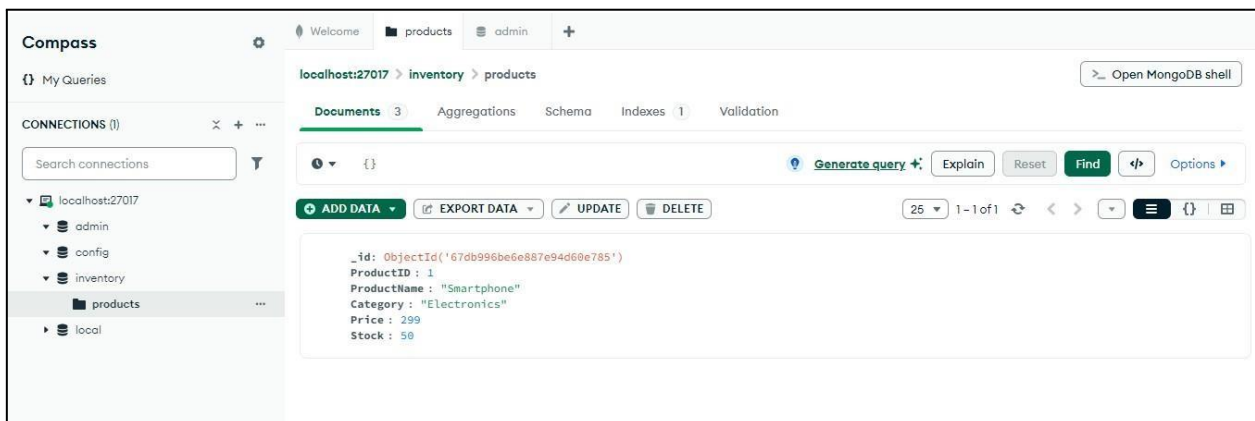
Insert Document

To collection `inventory.products`

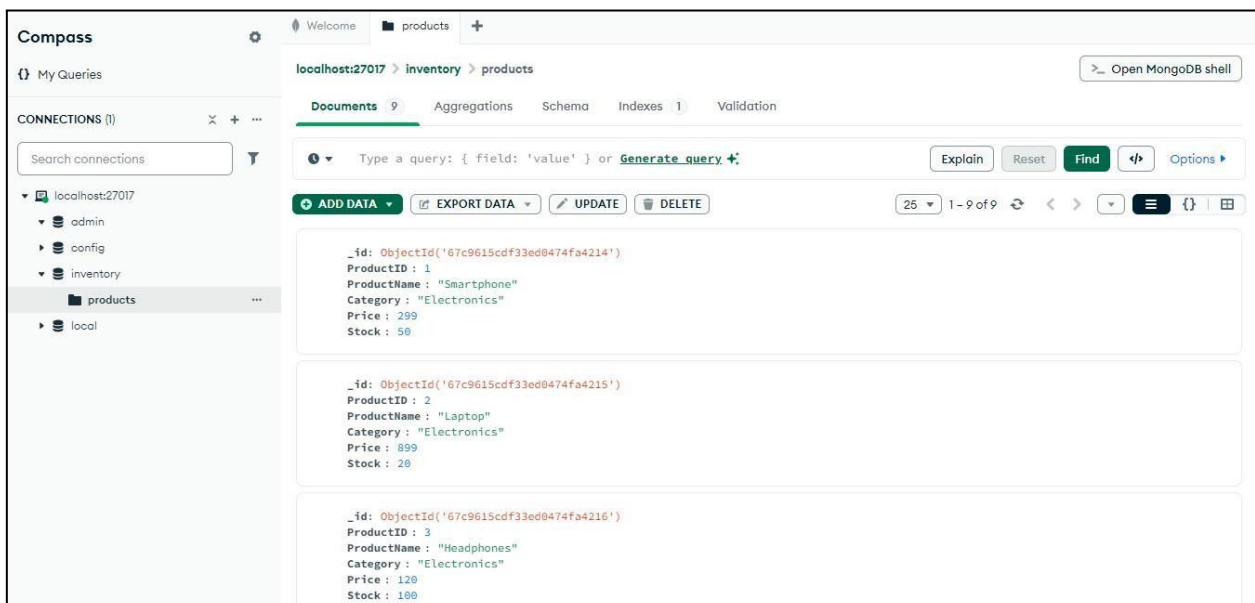
VIEW `{ }` `≡`

```
1 {  
2   "ProductID": 1,  
3   "ProductName": "Smartphone",  
4   "Category": "Electronics",  
5   "Price": 299,  
6   "Stock": 50  
7 }  
8
```

Cancel Insert



Added more data to the database -



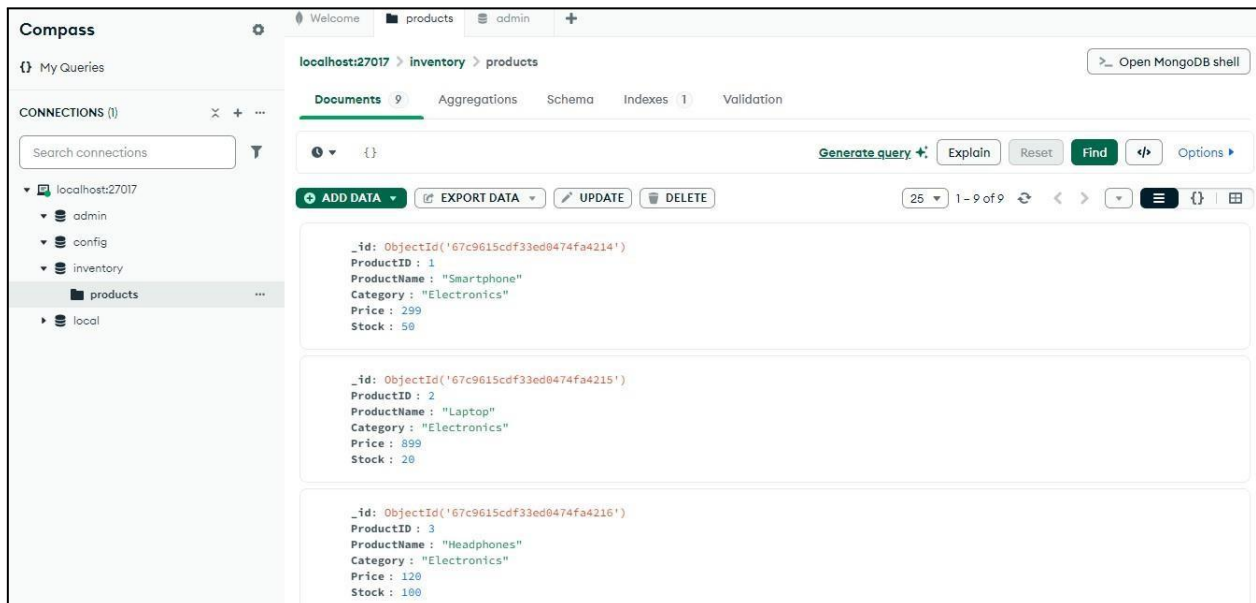
Read Data (Retrieve Documents)

1. Click on the `inventory` collection.
2. In the **"FILTER"** field, enter queries to retrieve data.

a) Get all products:

- Query:

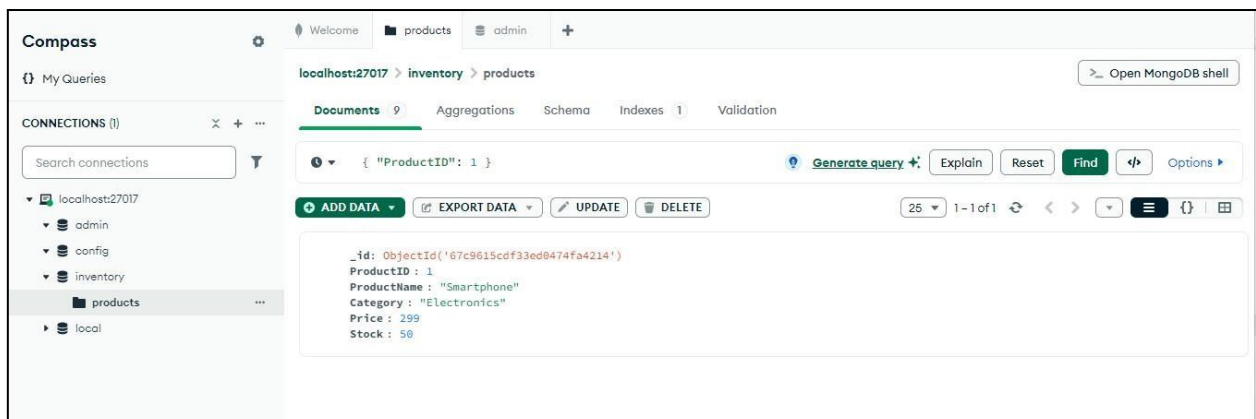
```
{ }
```



b) Get a specific product by `ProductID`:

- Query:

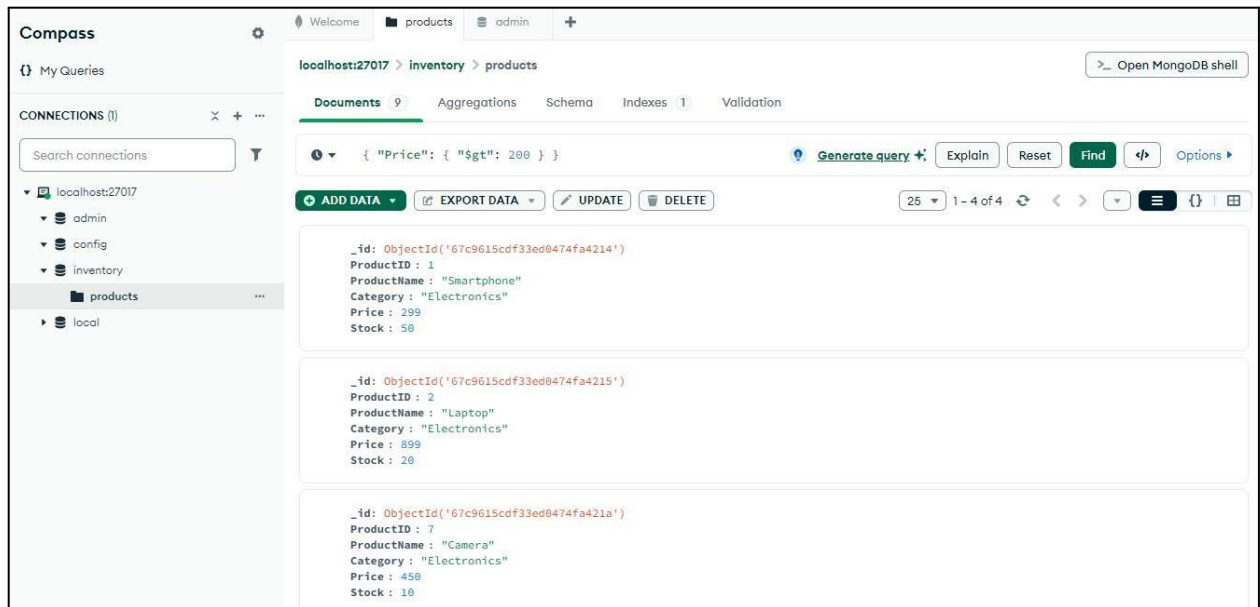
```
{ "ProductID": 1 }
```



c) Get products with price greater than 200:

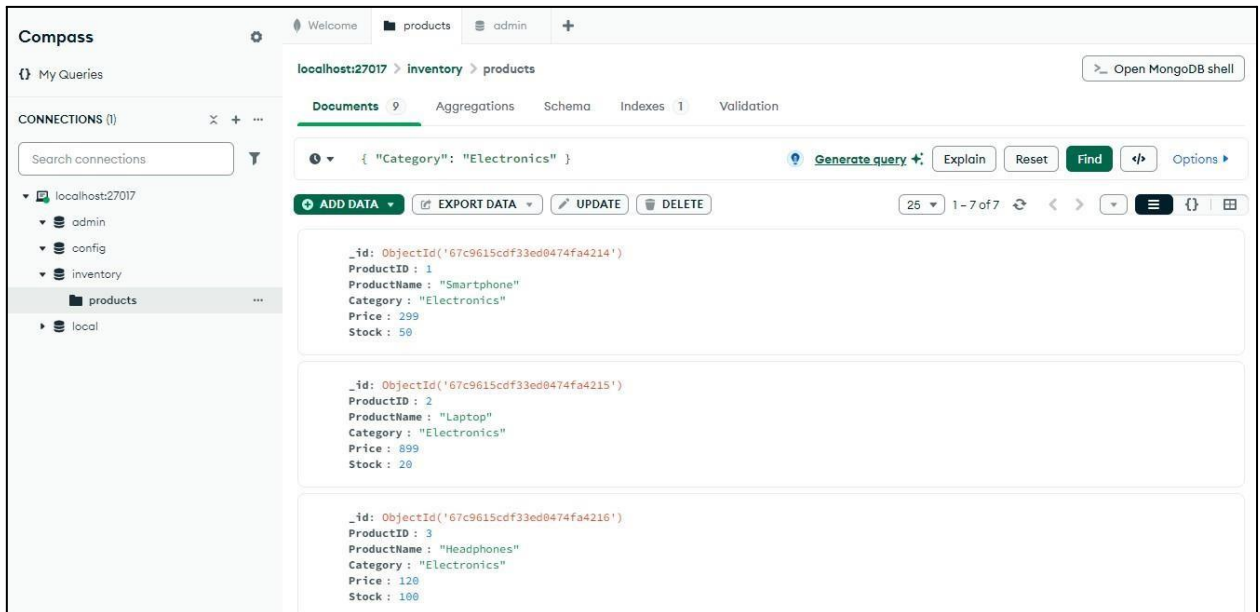
- Query:

```
{ "Price": { "$gt": 200 } }
```



d) Get all products in the "Electronics" category:

- Query:
`{ "Category": "Electronics" }`



Update Data

a) Update the price of a product:

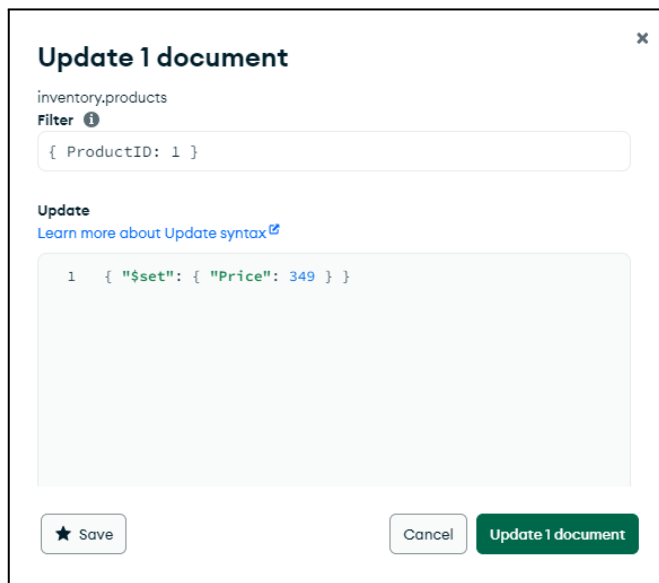
Filter Query (to find the product):

```
{ "ProductID": 1 }
```

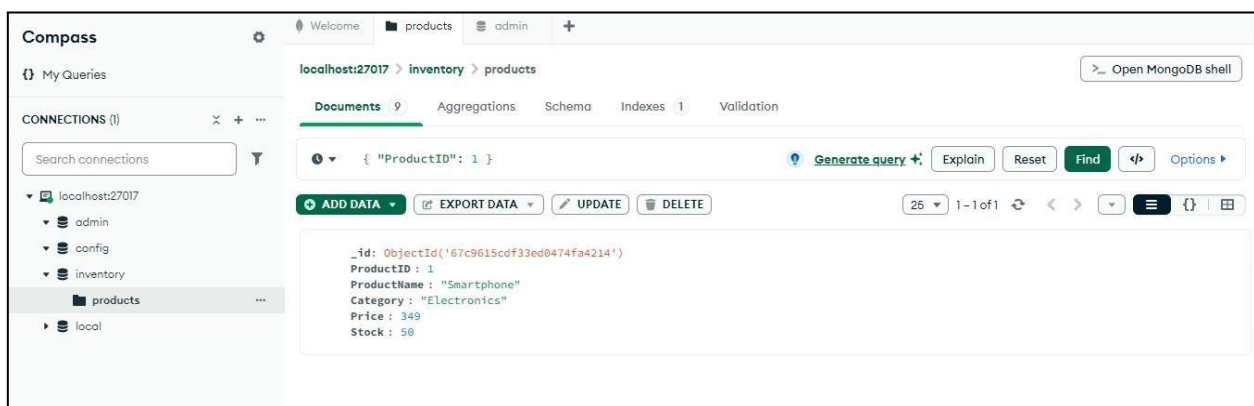
Update Query:

```
{ "$set": { "Price": 349 } }
```

- Click **"Update"**.



The screenshot shows the 'Update 1 document' dialog box in MongoDB Compass. The dialog is titled 'Update 1 document' and has a close button (X) in the top right corner. It shows the collection 'inventory.products'. Under the 'Filter' section, the query '{ ProductID: 1 }' is entered. Under the 'Update' section, the update query '1 { "\$set": { "Price": 349 } }' is entered. At the bottom, there are three buttons: 'Save' (with a star icon), 'Cancel', and 'Update 1 document' (in a green box).



b) Add a new field "Discount" to all products:

Filter Query:

```
{ "Category": "Electronics" }
```

Update Query:

```
{ "$set": { "Discount": true } }
```

- Click "Update Many".

Update 7 documents

inventory.products

Filter ⓘ
{ Category: 'Electronics' }

Update
[Learn more about Update syntax](#)

```
1 { "$set": { "Discount": true } }
```

★ Save

Cancel

Update 7 documents

Compass

My Queries

CONNECTIONS (1)

localhost:27017

admin

config

inventory

products

local

localhost:27017 > inventory > products

Documents 9 Aggregations Schema Indexes 1 Validation

{ "Category": "Electronics" }

Generate query Explain Reset Find Options

ADD DATA EXPORT DATA UPDATE DELETE

25 1 - 7 of 7

```
_id: ObjectId('67c9615cdf33ed0474fa4214')
ProductID: 1
ProductName: "Smartphone"
Category: "Electronics"
Price: 349
Stock: 50
Discount: true
```

```
_id: ObjectId('67c9615cdf33ed0474fa4215')
ProductID: 2
ProductName: "Laptop"
Category: "Electronics"
Price: 899
Stock: 20
Discount: true
```

```
_id: ObjectId('67c9615cdf33ed0474fa4216')
ProductID: 3
ProductName: "Headphones"
Category: "Electronics"
Price: 120
```

Delete Data

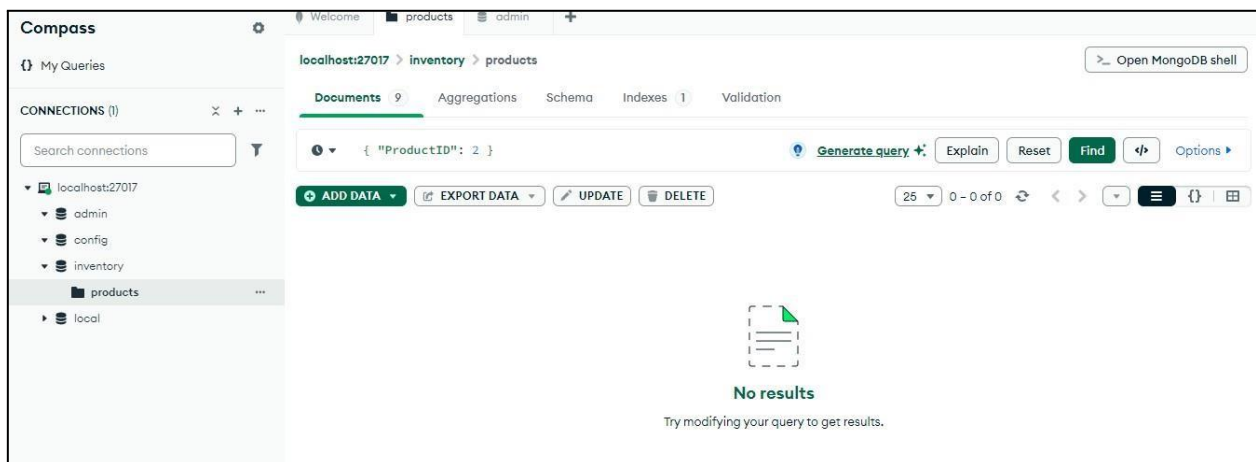
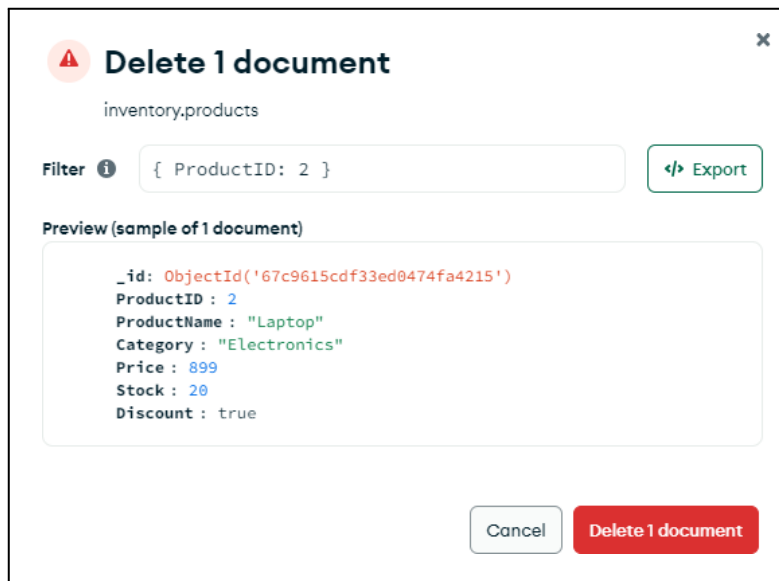
1. Click on the `inventory` collection.
2. Click "**FILTER**" and enter the query to find the document you want to delete.
3. Click "**DELETE**".

a) Delete a specific product:

Filter Query:

```
{ "ProductID": 2 }
```

- Click "Delete One".

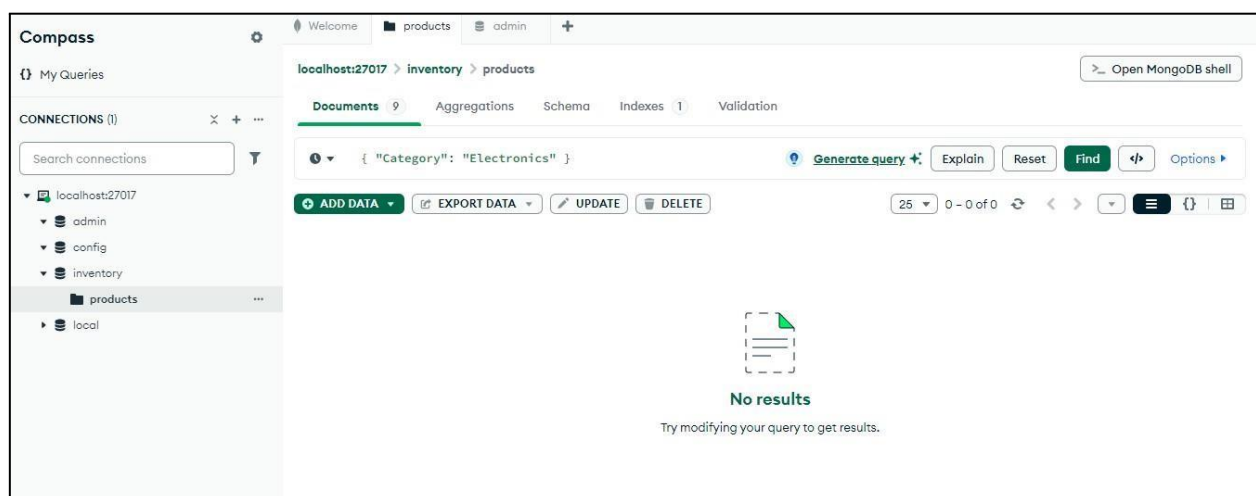
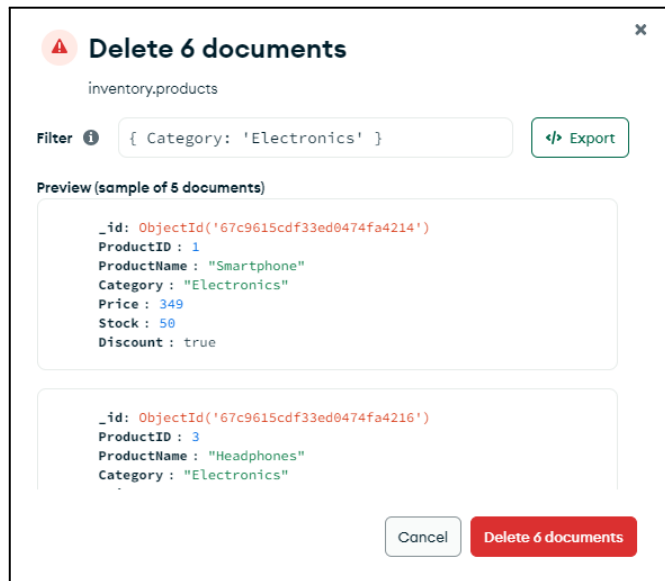


b) Delete all products in the "Electronics" category:

Filter Query:

```
{ "Category": "Electronics" }
```

- Click "Delete Many".



TYPE 2: Doing manually open the command prompt and enter mongosh and start inserting data .

```

Microsoft Windows [Version 10.0.22631.5839]
(c) Microsoft Corporation. All rights reserved.

C:\Users\PC>mongosh
Current Mongosh Log ID: 67e422c622d8596a43b71235
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.4.2
Using MongoDB:      7.0.18
Using Mongosh:       2.4.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2025-03-26T21:16:39.446+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> show dbs
admin   48.00 KiB
config 12.00 KiB
local   48.00 KiB
test> use inventory
switched to db inventory
inventory> db.createCollection("products")
{ ok: 1 }
inventory> db.products.insertMany([
...   { ProductID: 1, ProductName: "Laptop", Category: "Electronics", Price: 55000, Stock: 30 },
...   { ProductID: 2, ProductName: "Smartphone", Category: "Electronics", Price: 30000, Stock: 50 },
...   { ProductID: 3, ProductName: "Headphones", Category: "Electronics", Price: 2000, Stock: 100 },
...   { ProductID: 4, ProductName: "Tablet", Category: "Electronics", Price: 20000, Stock: 25 },
...   { ProductID: 5, ProductName: "Shoes", Category: "Clothing", Price: 3000, Stock: 40 },
...   { ProductID: 6, ProductName: "T-Shirt", Category: "Clothing", Price: 800, Stock: 60 },
...   { ProductID: 7, ProductName: "Pen", Category: "Stationery", Price: 50, Stock: 500 },
...   { ProductID: 8, ProductName: "Notebook", Category: "Stationery", Price: 100, Stock: 200 },
...   { ProductID: 9, ProductName: "Mouse", Category: "Electronics", Price: 800, Stock: 75 },
...   { ProductID: 10, ProductName: "Keyboard", Category: "Electronics", Price: 1500, Stock: 80 }
... ])
...

```

```

... ])
...
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67e423cb22d8596a43b71236'),
    '1': ObjectId('67e423cb22d8596a43b71237'),
    '2': ObjectId('67e423cb22d8596a43b71238'),
    '3': ObjectId('67e423cb22d8596a43b71239'),
    '4': ObjectId('67e423cb22d8596a43b7123a'),
    '5': ObjectId('67e423cb22d8596a43b7123b'),
    '6': ObjectId('67e423cb22d8596a43b7123c'),
    '7': ObjectId('67e423cb22d8596a43b7123d'),
    '8': ObjectId('67e423cb22d8596a43b7123e'),
    '9': ObjectId('67e423cb22d8596a43b7123f')
  }
}
inventory> db.products.find({ Category: "Electronics" }).pretty()
[
  {
    _id: ObjectId('67e423cb22d8596a43b71236'),
    ProductID: 1,
    ProductName: 'Laptop',
    Category: 'Electronics',
    Price: 55000,
    Stock: 30
  },
  {
    _id: ObjectId('67e423cb22d8596a43b71237'),
    ProductID: 2,
    ProductName: 'Smartphone',
    Category: 'Electronics',
    Price: 30000,
    Stock: 50
  },
  {
    _id: ObjectId('67e423cb22d8596a43b71238'),
    ProductID: 3,
    ProductName: 'Headphones',
    Category: 'Electronics',
    Price: 2000,

```

CONCLUSION

Through this experiment, we successfully performed **CRUD operations** in **MongoDB**, including **creating a database**, **inserting documents**, **querying data**, **updating records**, and **deleting entries**. We also explored filtering data, sorting, and aggregation queries.

MongoDB's document-oriented structure and flexible schema make it an ideal choice for handling large-scale, unstructured data in real-world applications.